MODUL WEEK 12

DATABASE SYSTEM

In this session, we're going to make a simple NoSQL database system using MongoDB to store users' data:

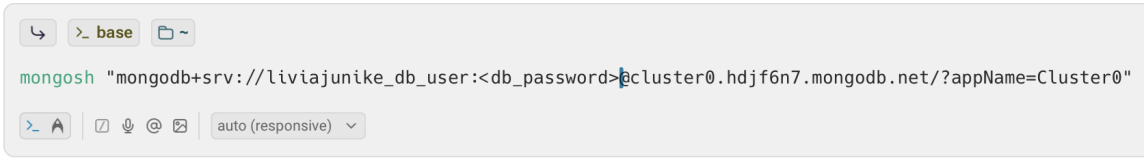
- **id**: object ID
- **fullName**: string
- **email**: string
- **age**: number
- **phoneNumber**: string
- **registeredAt**: date

To achieve this, you'll need to follow the following prerequisites and instructions below.

PREREQUISITES:

1. Mongosh
2. Connection string from MongoDB Atlas Dashboard (https://cloud.mongodb.com/v2/)

   If you've already installed the Mongosh and copy paste your string connection to your terminal, the terminal should be looked like this (not 100% same, but similar):



   Note: you should change the **<db_password>** to your database user's password. The output should look like this after you press enter:

```
Current Mongosh Log ID: 68fdcce5e4db4bbd4d387c5d
Connecting to:          mongodb+srv://<credentials>@cluster0.hdjf6n7.mongodb.net/?appName=Cluster0
Using MongoDB:          8.0.15
Using Mongosh:          2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

Atlas atlas-o2r516-shard-0 [primary] test> ⌘I to generate
```

3. Create a New Database

   Create a new database from your Mongosh named "**LAB_WEEK12**" (Not sure how to create a new database? See: Create A MongoDB Database)

ASSIGNMENT

After you complete this module, you'll need to submit the code in a **.txt file**, and name it with the following format: **NIM_FullName_Week12.txt**

1. **Inserting Documents: insertOne, insertMany**
   a. insertOne

      To insert new data to the MongoDB database, we can use `db.<databaseName>.insertOne()`. Here's the example:

```
Atlas atlas-o2r516-shard-0 [primary] test> use LAB_WEEK12
switched to db LAB_WEEK12
Atlas atlas-o2r516-shard-0 [primary] LAB_WEEK12> db.LAB_WEEK12.insertOne({
| fullName: "Livia Junike",
| email: "livia.junike@student.umn.ac.id",
| age: 19,
| phoneNumber: "081387651024",
| registeredAt: new Date()})
{
  acknowledged: true,
  insertedId: ObjectId('68fdce69e4db4bbd4d387c5e')
}
```

      If you try to view all of the stored data:

```
[
  {
    _id: ObjectId('68fdce69e4db4bbd4d387c5e'),
    fullName: 'Livia Junike',
    email: 'livia.junike@student.umn.ac.id',
    age: 19,
    phoneNumber: '081387651024',
    registeredAt: ISODate('2025-10-26T07:31:53.640Z')
  }
]
```

      Do you still remember how to list all of the stored data via Mongosh?

      TODO: Insert 2 more data with `db.databaseName.insertOne()`!

   b. insertMany

      We can use `db.<databaseName>.insertMany()` to insert more than 1 data at a time. Example (you don't have to copy this):

```
Atlas atlas-o2r516-shard-0 [primary] LAB_WEEK12> db.LAB_WEEK12.insertMany([
|   {
|     fullName: "Budi Santoso",
|     email: "budi.santoso@example.com",
|     age: 34,
|     phoneNumber: "081122334455",
|     registeredAt: new Date()
|   },
|   {
|     fullName: "Citra Lestari",
|     email: "citra.lestari@example.com",
|     age: 25,
|     phoneNumber: "085566778899",
|     registeredAt: new Date()
|   }])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68fdfbaee4db4bbd4d387c5f'),
    '1': ObjectId('68fdfbaee4db4bbd4d387c60')
  }
}
```

Now, with the same data structure, insert 3 new data with `db.<databaseName>.insertMany()`!

2. Updating Document: updateOne

Imagine after inserting all of the data to the database, your supervisor wants you to update the user's data named "Livia Junike" (for example):

```
Atlas atlas-o2r516-shard-0 [primary] LAB_WEEK12> db.LAB_WEEK12.updateOne({
| fullName: "Livia Junike"},
| {"$set": {age: 20}
| })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

In this example, we're trying to update Livia's age from 19 to 20. Another update syntax that can be used to update data:

`updateMany(filter, modifier):` update more than 1 data at a time

`replaceOne(filter, newData):` replacing the old data with the new one (adding new column is also possible)

TODO: try the `updateOne()` for 2 data and `replaceOne()` for 1 data in your document!

3. Removing Document: drop

To permanently remove a data, you can use `db.<databaseName>.deleteOne():`

```
Atlas atlas-o2r516-shard-0 [primary] LAB_WEEK12> db.LAB_WEEK12.find()
[
  {
    _id: ObjectId('68fdce69e4db4bbd4d387c5e'),
    fullName: 'Livia Junike',
    email: 'livia.junike@student.umn.ac.id',
    age: 20,
    phoneNumber: '081387651024',
    registeredAt: ISODate('2025-10-26T07:31:53.640Z')
  },
  {
    _id: ObjectId('68fdfbaee4db4bbd4d387c5f'),
    fullName: 'Budi Santoso',
    email: 'budi.santoso@example.com',
    age: 34,
    phoneNumber: '081122334455',
    registeredAt: ISODate('2025-10-26T10:45:02.829Z')
  },
  {
    _id: ObjectId('68fdfbaee4db4bbd4d387c60'),
    fullName: 'Citra Lestari',
    email: 'citra.lestari@example.com',
    age: 25,
    phoneNumber: '085566778899',
    registeredAt: ISODate('2025-10-26T10:45:02.829Z')
  }
]
```

```
Atlas atlas-o2r516-shard-0 [primary] LAB_WEEK12> var database = db.LAB_WEEK12

Atlas atlas-o2r516-shard-0 [primary] LAB_WEEK12> database.deleteOne({fullName: "Citra Lestari"})
{ acknowledged: true, deletedCount: 1 }
Atlas atlas-o2r516-shard-0 [primary] LAB_WEEK12> database.find()
[
  {
    _id: ObjectId('68fdce69e4db4bbd4d387c5e'),
    fullName: 'Livia Junike',
    email: 'livia.junike@student.umn.ac.id',
    age: 20,
    phoneNumber: '081387651024',
    registeredAt: ISODate('2025-10-26T07:31:53.640Z')
  },
  {
    _id: ObjectId('68fdfbaee4db4bbd4d387c5f'),
    fullName: 'Budi Santoso',
    email: 'budi.santoso@example.com',
    age: 34,
    phoneNumber: '081122334455',
    registeredAt: ISODate('2025-10-26T10:45:02.829Z')
  }
]
```

To reduce typing repetition, I store the `db.LAB_WEEK12` to the `database` variable.

Tired of doing your unpaid internship? You can simply drop the document with `db.<databaseName>.drop()` or `db.<databaseName>.deleteMany({})`!

(Please, don't try this in a real-life internship.)

TODO: try to delete 3 data from your document using the `db.<databaseName>.deleteMany()` command!

4. **Upsert**

   As the name says, upsert is a combination of ***update*** and ***insert***. What are the necessities—you might think? Well, without upsert, usually we would need to run separate queries, the first is to find whether the document exist:

   ```
   const document = db.<collectionName>.findOne({
   uniqueIndexedField: value })
   ```

   > *the findOne() method returns either the first document found matching the filter (in this case is uniqueIndexedField), or null—if none was found.*

   Then, the returned value will be checked something like:

   ```
   if (document) {
         db.<collectionName>.updateOne({ uniqueIndexedField: value },
         { $set: { someField: value } })
   } else {
         db.<collectionName>.insertOne({ uniqueIndexedField: value,
         someField: value })
   }
   ```

   Very ineffective, don't you think? Without checking whether the document exists or not, the insert or update operation could fail. That's why ***upsert*** is here to provide that 2-in-1 solution! With ***upsert***:

   > *If a **document matching the filters doesn't exist** in the collection, **a new one** will be inserted/created (with fields merged from the filter and update parameters), **else** it will **update the existing document found** and its parameters accordingly.*

   Upsert can be used within multiple methods such as:

- *updateOne(<filter>, <update>, <options>),*

- *updateMany(<filter>, <update>, <options>),*
- *replaceOne(<filter>, <replacement>, <options>),*
- *findOneAndUpdate(<filter>, <update>, <options>),*
- *findOneAndReplace(<filter>, <replacement>, <options>).*

For example, let's add a new **`user`** to our collection, the values are up to you to decide, just make sure the email value doesn't exist yet to avoid duplicates, keeping it uniquely indexed.

```
LAB_WEEK12> db.LAB_WEEK12.insertOne({ fullName: "Sandy Bonfilio Y
uvens", email: "sandy.bonfilio@student.umn.ac.id", age: 20, phone
Number: "089687781729", registeredAt: new Date() })
{
  acknowledged: true,
  insertedId: ObjectId('690acaa698ed38792463b112')
}
LAB_WEEK12> db.LAB_WEEK12.find({ email: "sandy.bonfilio@student.u
mn.ac.id" })
[
  {
    _id: ObjectId('690acaa698ed38792463b112'),
    fullName: 'Sandy Bonfilio Yuvens',
    email: 'sandy.bonfilio@student.umn.ac.id',
    age: 20,
    phoneNumber: '089687781729',
    registeredAt: ISODate('2025-11-05T03:55:18.842Z')
  }
]
```

> *When working with **upserts**, it can create duplicate documents—unless a unique index exists. Hence, an existing unique indexed field is preferred when working with it to avoid mis-configured filter conditions. If you'd like to read more detailed information about multiple upserts and duplicate values, feel free to read this documentation.*

Now, in the context of the current database structure, ***upsert*** can be used to help us handle cases where data changes due to the user updating his/her profile as time goes by, or the needs of adding new user profiles. Let's say the user we've just inserted before, updated his/her **`phoneNumber`** to a new value, and perhaps your app's latest update has allowed users to fill in a new **`displayName`** field. We can use and set the ***upsert*** property to true in the <options> parameter like this:

```
LAB_WEEK12> db.LAB_WEEK12.updateOne({ email: "sandy.bonfilio@stud
ent.umn.ac.id" }, { $set: { phoneNumber: "089637073749", displayN
ame: "sandyb" }, $setOnInsert: { fullName: "Sandy Bonfilio Yuvens
", age: 20, registeredAt: new Date() } }, { upsert: true })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
LAB_WEEK12> db.LAB_WEEK12.find({ email: "sandy.bonfilio@student.u
mn.ac.id" }).pretty()
[
  {
    _id: ObjectId('690acaa698ed38792463b112'),
    fullName: 'Sandy Bonfilio Yuvens',
    email: 'sandy.bonfilio@student.umn.ac.id',
    age: 20,
    phoneNumber: '089637073749',
    registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
    displayName: 'sandyb'
  }
]
```

The filtered field for the document to be operated on is the `email` *field (we assume it's uniquely indexed), and in the <update> parameter, we are defining two update operators (view more update operators here), which are* **`$set`** *and* **`$setOnInsert`**. **`$setOnInsert`** *will set the values of each field defined in the query* ***if*** *an update operation* ***results*** *in an insert of a document (doesn't exist yet), and will not have effect on a successful update operation (document exists). In this case, since the user exists, the fields* `phoneNumber` *and* `displayName` *are the ones that will have its values changed. Thus, the* `upsertedCount` *says 0.*

TODO: try to upsert 2 documents (or more), with one of the documents not existing yet in your collection, using the **`upsert: true`** option!

5. **Updating Multiple Document: updateMany**

   updateMany() is a dedicated method introduced along with the updateOne() method, to address some issues found while using the deprecated update() method (learn more about updateMany here). Before it existed, developers had to explicitly specify the `multi: true` option to achieve modifying multiple

documents. Let's see it in action, say that we want to add a new field **`role`** for all the users in the database. We can perform an updateMany() like shown below:

```
LAB_WEEK12> db.LAB_WEEK12.updateMany({ role: { $exists: false } }, { $
set: { role: "user" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 19,
  modifiedCount: 19,
  upsertedCount: 0
}
```

> *To ensure the filter conditions are correct, you can modify it using the available filter/query operators such as the* **`$exists`** *operator used above to filter for documents with no field named* **`role`** *yet, or other operators that you can learn more about here.*

```
{
  _id: ObjectId('690acaa698ed38792463b112'),
  fullName: 'Sandy Bonfilio Yuvens',
  email: 'sandy.bonfilio@student.umn.ac.id',
  age: 20,
  phoneNumber: '089637073749',
  registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
  displayName: 'sandyb',
  role: 'user'
},
```

You can also perform other operations such as **`$unset`** to remove a certain field, based on your filter.

```
LAB_WEEK12> db.LAB_WEEK12.updateMany({}, { $unset: { role: "" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 19,
  modifiedCount: 19,
  upsertedCount: 0
}
```

```
{
  _id: ObjectId('690acaa698ed38792463b112'),
  fullName: 'Sandy Bonfilio Yuvens',
  email: 'sandy.bonfilio@student.umn.ac.id',
  age: 20,
  phoneNumber: '089637073749',
  registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
  displayName: 'sandyb'
},
```

TODO: try to use the updateMany() method to set a new field called **`isOverLegalAge`** with a value of "Yes", that will have a filter on the **`age`** field which has a value of 21 years or over, otherwise "No".

```
{
  _id: ObjectId('690acaa698ed38792463b112'),
  fullName: 'Sandy Bonfilio Yuvens',
  email: 'sandy.bonfilio@student.umn.ac.id',
  age: 20,
  phoneNumber: '089637073749',
  registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
  displayName: 'sandyb',
  isOverLegalAge: 'No'
},
{
  _id: ObjectId('690ada54a86e443a9d966c78'),
  email: 'sandy@student.umn.ac.id',
  age: 23,
  displayName: 'sandal',
  fullName: 'Sandal Cheeks',
  phoneNumber: '089676767676',
  registeredAt: ISODate('2025-11-05T05:02:12.930Z'),
  isOverLegalAge: 'Yes'
}
```

***NOTE**: To achieve this, a few approaches can be done, you are free to choose! Performing separate operations based on the filter condition works well when working with small datasets like these. If you are aiming to learn more about complex transformations that are more flexible and expressive to use, you can learn about aggregation pipelines and updates with aggregation pipelines, especially using the **`$set`** and **`$cond`** operators, or any other operators available. We will focus on it next week though!*

6. **Embedding for Locality, Atomicity, and Isolation**

When dealing with data modeling or schema design, decisions between embedding and referencing comes to mind. Embedding is a direct approach as its name suggests, nesting documents inside the parent document. Most common use cases to use embedding are:

- One-to-one/one-to-few relationships (relatively small sized),
- Data that are frequently accessed/updated together, and highly dependent on each other,
- No frequent independent updates,

For example, a one-to-one relationship between the user and address:

```
LAB_WEEK12> db.LAB_WEEK12.updateOne({ email: "sandy.bonfilio@student.u
mn.ac.id" }, { $set: { address: { street: "Jl. Scientia Boulevard", ci
tyOrRegency: "Tangerang", zipCode: "15810" } } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
LAB_WEEK12> db.LAB_WEEK12.find({ email: "sandy.bonfilio@student.umn.ac
.id" })
[
  {
    _id: ObjectId('690acaa698ed38792463b112'),
    fullName: 'Sandy Bonfilio Yuvens',
    email: 'sandy.bonfilio@student.umn.ac.id',
    age: 20,
    phoneNumber: '089637073749',
    registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
    displayName: 'sandyb',
    isOverLegalAge: 'No',
    address: {
      street: 'Jl. Scientia Boulevard',
      cityOrRegency: 'Tangerang',
      zipCode: '15810'
    }
  }
]
```

This is fine as we assumed that the users are only allowed to have a single address. For one-to-few relationships, it is still okay, as long as the number of addresses each user has aren't that many or will not grow in size indefinitely:

```
LAB_WEEK12> db.LAB_WEEK12.updateOne({ email: "sandy.bonfilio@student.u
mn.ac.id" }, { $unset: { address: {} }, $push: { addresses: { $each: [
{ street: "Jl. Scientia Boulevard", cityOrRegency: "Tangerang", zipCod
e: "15810", type: "work" }, { street: "Jl. Benteng Makasar", cityOrReg
ency: "Tangerang", zipCode: "15111", type: "home" }] } } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
LAB_WEEK12> db.LAB_WEEK12.find({ email: "sandy.bonfilio@student.umn.ac
.id" })
[
  {
    _id: ObjectId('690acaa698ed38792463b112'),
    fullName: 'Sandy Bonfilio Yuvens',
    email: 'sandy.bonfilio@student.umn.ac.id',
    age: 20,
    phoneNumber: '089637073749',
    registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
    displayName: 'sandyb',
    isOverLegalAge: 'No',
    addresses: [
      {
        street: 'Jl. Scientia Boulevard',
        cityOrRegency: 'Tangerang',
        zipCode: '15810',
        type: 'work'
      },
      {
        street: 'Jl. Benteng Makasar',
        cityOrRegency: 'Tangerang',
        zipCode: '15111',
        type: 'home'
      }
    ]
  }
]
```

But you get the idea—if it's applied to other examples of relationships or use cases—for example, posts, likes or comments in a social media app, user reviews in an e-commerce app with a lot of users, or anything that might grow in size indefinitely over the time as well as having a complex document structure, it could get too troublesome and might cause a crash due to the maximum BSON file size allowed.

Take a look at this example, suppose we want to keep track on posts that each user has liked, embedded inside the document of each user:

```
LAB_WEEK12> db.users.bulkWrite([
... { updateOne: { filter: { email: "sandy.bonfilio@student.umn.ac.id" }, up
date: { $addToSet: { likedPosts: { $each: [{ postId: "P001", caption: "Disco
ver, Develop, Dominate!", postedBy: "ppif.umn", postedAt: new Date("2025-08-
12") }, { postId: "P105", caption: "Explore Tomorrow, Create Today!", posted
By: "byte_umn", postedAt: new Date("2025-11-05") }] } } } } },
... { updateOne: { filter: { email: "sandy@student.umn.ac.id" }, update: { $
addToSet: { likedPosts: { postId: "P105", caption: "Explore Tomorrow, Create
 Today!", postedBy: "byte_umn", postedAt: new Date("2025-11-05") } } } } }
... ])
{
  acknowledged: true,
  insertedCount: 0,
  insertedIds: {},
  matchedCount: 2,
  modifiedCount: 2,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {}
}
```

In the code executed above, I used bulkWrite() to allow me perform multiple write operations (in this case, updates) at once in a single call. Along with some useful operators such as **`$addToSet`***—used to add a certain value to an array field, only if the value doesn't exist yet in that array and* **`$each`***—which is useful when combined with* **`$addToSet`** *or* **`$push`** to add multiple elements in a single operation, especially when *working with array structures like these.*

```
{
  _id: ObjectId('690ada54a86e443a9d966c78'),
  email: 'sandy@student.umn.ac.id',
  age: 23,
  displayName: 'sandal',
  fullName: 'Sandal Cheeks',
  phoneNumber: '089676767676',
  registeredAt: ISODate('2025-11-05T05:02:12.930Z'),
  isOverLegalAge: 'Yes',
  addresses: [
    {
      street: 'Jl. Bikini Bottom',
      cityOrRegency: 'Undersea',
      zipCode: '11111',
      type: 'home'
    }
  ],
  roles: [ 'user' ],
  likedPosts: [
    {
      postId: 'P105',
      caption: 'Explore Tomorrow, Create Today!',
      postedBy: 'byte_umn',
      postedAt: ISODate('2025-11-05T00:00:00.000Z')
    }
  ]
}
```

Then, to access or query data inside an embedded document, we can use the dot notation, specified with the field name inside quotation marks. For example, querying all the users that have liked a post with an ID of “P105”:

```
LAB_WEEK12> db.users.find({ "likedPosts.postId": "P105" }).pretty()
```

Now, do you notice anything that might cause problems with the current structure? *Data duplication*! If multiple users like the same post, its details are also duplicated across many user documents, since it’s embedded directly. When a certain field of the post changes, let’s say the user that posted the post edits the caption, we must then update it in every document where it’s embedded:

```
LAB_WEEK12> db.users.updateMany({ "likedPosts.postId": "P105" }, { $set: { "
likedPosts.$[element].caption": "Bringing Your Tech Experience 2025" } }, {
arrayFilters: [ { "element.postId": "P105" } ] })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

```
likedPosts: [
  {
    postId: 'P001',
    caption: 'Discover, Develop, Dominate!',
    postedBy: 'ppif.umn',
    postedAt: ISODate('2025-08-12T00:00:00.000Z')
  },
  {
    postId: 'P105',
    caption: 'Bringing Your Tech Experience 2025',
    postedBy: 'byte_umn',
    postedAt: ISODate('2025-11-05T00:00:00.000Z')
  }
]
```

Other problems that you might’ve foreseen:

- The size of the likedPosts array might increase over time and eventually exceed the file size limits since it holds all the fields of the posts, and
- If a certain post changes, you have to update it across multiple users and lose atomicity, and there are possibilities that some users might still have the old values if an error occurs during the update operations.

Positional Operators ($)

You might've noticed that there is a new syntax using the ($) symbol in the operation above, it is called the positional operators, which allows modifications inside an array of embedded documents easily.

- Positional update operator ($), returns/references the first element in the array that matches the filter/query condition.
- All positional operator ($[]), which modify all elements in the specified array field.
- Filtered positional operator ($[<identifier>]), similar to the all positional operator, but modifies all the elements that match the specified arrayFilters conditions.

TODO: embed a **`posts`** field inside the users' documents, which lists all the posts each user currently has. You can populate or modify its properties/sub-fields according to your liking. Just make sure that there is a unique ID for each post and no duplicates, as well as ensuring the related posts inside likedPosts or any other fields (if there are any) have been adjusted accordingly! Then display the newly embedded documents:

```
LAB_WEEK12> db.users.find({ email: "sandy.bonfilio@student.umn.ac.id" }, { _id: 1,
email: 1, displayName: 1, fullName: 1, likedPosts: 1, posts: 1 })
[
  {
    _id: ObjectId('690acaa698ed38792463b112'),
    fullName: 'Sandy Bonfilio Yuvens',
    email: 'sandy.bonfilio@student.umn.ac.id',
    displayName: 'sandyb',
    likedPosts: [
      {
        postId: 'P001',
        caption: 'Discover, Develop, Dominate!',
        postedBy: 'ppif.umn',
        postedAt: ISODate('2025-08-12T00:00:00.000Z')
      },
      {
        postId: 'P105',
        caption: 'Bringing Your Tech Experience 2025',
        postedBy: 'byte_umn',
        postedAt: ISODate('2025-11-05T00:00:00.000Z')
      }
    ],
    posts: [
      {
        postId: 'P189',
        caption: 'admin admin admin',
        postedBy: 'sandyb',
        postedAt: ISODate('2025-09-18T00:00:00.000Z')
      }
    ]
  }
]
```

7. **Referencing for Flexibility**

Some common use cases of referencing are:

- Having one-to-many (with an unbounded maximum limit)/many-to-many relationships,
- Avoiding data duplications and keeping the file size well below the maximum file size allowed,
- Aiming for the ability to access related data independently, or
- Gaining more flexibility when the data change often.

What about the trade-off? *Performance*. It could be resource-intensive, or slower in performance depending on the size of the datasets, as well as your queries approaches. Generally, **`$lookup`** is convenient when wanting to do complex queries. Another common approach is to perform multiple queries which can run asynchronously/parallely..

Moving on, let's now take a look at our existing **`users`** document structure:

```
LAB_WEEK12> db.users.find({ email: "sandy.bonfilio@student.umn.ac.id" })
[
  {
    _id: ObjectId('690acaa698ed38792463b112'),
    fullName: 'Sandy Bonfilio Yuvens',
    email: 'sandy.bonfilio@student.umn.ac.id',
    age: 20,
    phoneNumber: '089637073749',
    registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
    displayName: 'sandyb',
    isOverLegalAge: 'No',
    addresses: [
      {
        street: 'Jl. Scientia Boulevard',
        cityOrRegency: 'Tangerang',
        zipCode: '15810',
        type: 'work'
      },
      {
        street: 'Jl. Benteng Makasar',
        cityOrRegency: 'Tangerang',
        zipCode: '15111',
        type: 'home'
      }
    ],
    likedPosts: [
      {
        postId: 'P001',
        caption: 'Discover, Develop, Dominate!',
        postedBy: 'ppif.umn',
        postedAt: ISODate('2025-08-12T00:00:00.000Z')
      },
      {
        postId: 'P105',
        caption: 'Bringing Your Tech Experience 2025',
        postedBy: 'byte_umn',
        postedAt: ISODate('2025-11-05T00:00:00.000Z')
      }
    ],
    accountDetails: { followersCount: 170, followingsCount: 289, postsCount: 3 },
    posts: [
      {
        postId: 'P189',
        caption: 'admin admin admin',
        postedBy: 'sandyb',
        postedAt: ISODate('2025-09-18T00:00:00.000Z')
      }
    ]
  }
]
```

For practice purposes, we are going to migrate only the **`posts`** field from embedded into a separate collection. You can migrate them with plenty of approaches. The first one is using a manual loop and upserts:

```
LAB_WEEK12> db.users.find({ posts: { $exists: 1 } }).forEach(function (user) { user
.posts.forEach(function (post) { db.posts.updateOne({ _id: post.postId }, { $set: {
 caption: post.caption, postedBy: user._id, postedAt: post.postedAt } }, { upsert:
true }); }); db.users.updateOne({ _id: user._id }, { $unset: { posts: "" } }) });
```

Performance wise, it is considered slow if you have a large dataset, and no atomicity achieved here, since if the operation stopped midway, it might result in a data inconsistency.

The next approach is using the aggregate() method with a follow-up operation on **`$unset`** which is generally faster, especially for larger datasets, and it achieves atomicity. **But, since these topics will be discussed more thoroughly in the next week, you don't need to worry much about it, yet. Just follow along and keep it in mind, so that you know the abilities to perform operations such as these simple ones!**

```
LAB_WEEK12> db.users.aggregate([ { $unwind: "$posts" }, { $project: { _id: "$posts.
postId", caption: "$posts.caption", postedBy: "$_id", postedAt: "$posts.postedAt" }
 }, { $merge: { into: "posts", whenMatched: "replace", whenNotMatched: "insert" } }
 ])
```

```
LAB_WEEK12> db.users.updateMany({ posts: { $exists: 1 } }, { $unset: { posts: "" }
})
```

> *The* **`$unwind`** *stage is used to deconstruct elements in an array from the input document, which then outputs a document for each of them. Then the* **`$project`** *stage will handle all the necessary fields used to structure/format a new document (*`_id`*,* `caption`*,* `postedBy` *and* `postedAt`*), that were passed by the* `$unwind` *stage, which then will be passed onto the* **`$merge`** *stage, marking the end of the pipelining process, and then writing it to the destined collection. With* `$merge`*'s* `whenMatched` *and* `whenNotMatched` *fields, we can control whether the resulting document should replace an existing one in the target collection or be inserted as a new one if it doesn't exist yet.*

Now, your user document should not have the embedded **`posts`** field anymore:

```
{
  _id: ObjectId('690ada54a86e443a9d966c78'),
  email: 'sandy@student.umn.ac.id',
  age: 23,
  displayName: 'sandal',
  fullName: 'Sandal Cheeks',
  phoneNumber: '089676767676',
  registeredAt: ISODate('2025-11-05T05:02:12.930Z'),
  isOverLegalAge: 'Yes',
  addresses: [
    {
      street: 'Jl. Bikini Bottom',
      cityOrRegency: 'Undersea',
      zipCode: '11111',
      type: 'home'
    }
  ],
  roles: [ 'user' ],
  likedPosts: [
    {
      postId: 'P105',
      caption: 'Bringing Your Tech Experience 2025',
      postedBy: 'byte_umn',
      postedAt: ISODate('2025-11-05T00:00:00.000Z')
    }
  ]
```

And a new **`posts`** collection should appear:

```
LAB_WEEK12> db.posts.find().pretty()
[
  {
    _id: 'P189',
    caption: 'admin admin admin',
    postedAt: ISODate('2025-09-18T00:00:00.000Z'),
    postedBy: ObjectId('690acaa698ed38792463b112')
  },
  {
    _id: 'P001',
    caption: 'Discover, Develop, Dominate!',
    postedAt: ISODate('2025-08-12T00:00:00.000Z'),
    postedBy: ObjectId('690c6ab098ed38792463b117')
  },
  {
    _id: 'P105',
    caption: 'Bringing Your Tech Experience 2025',
    postedAt: ISODate('2025-11-05T00:00:00.000Z'),
    postedBy: ObjectId('690c6ab098ed38792463b118')
  },
  {
    _id: 'P101',
    caption: '#TGIF',
    postedBy: ObjectId('690ada54a86e443a9d966c78'),
    postedAt: ISODate('2025-11-03T17:00:00.000Z')
  },
  {
    _id: 'P895',
    caption: "Infinite's Registration Is Now Open!",
    postedBy: ObjectId('690c6ab098ed38792463b118'),
    postedAt: ISODate('2025-11-05T00:00:00.000Z')
  }
]
```

Now, let's start querying a basic **$lookup**:

```
LAB_WEEK12> db.posts.aggregate([
... { $lookup: { from: "users", localField: "postedBy", foreignField: "_id", as: "a
uthor" } }
... ])
[
  {
    _id: 'P189',
    caption: 'admin admin admin',
    postedAt: ISODate('2025-09-18T00:00:00.000Z'),
    postedBy: ObjectId('690acaa698ed38792463b112'),
    author: [
      {
        _id: ObjectId('690acaa698ed38792463b112'),
        fullName: 'Sandy Bonfilio Yuvens',
        email: 'sandy.bonfilio@student.umn.ac.id',
        age: 20,
        phoneNumber: '089637073749',
        registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
        displayName: 'sandyb',
        isOverLegalAge: 'No',
        addresses: [
          {
            street: 'Jl. Scientia Boulevard',
            cityOrRegency: 'Tangerang',
            zipCode: '15810',
            type: 'work'
          },
```

It outputs each post's author details, which is retrieved by joining the **`users`** and **`posts`** collections based on the specified **`localField`** and **`foreignField`** inside the **`$lookup`** operator. Now, if you notice, since we assumed that there will only be one and only one author for each post, we can use the **`$unwind`** operator to remove the array structure and flatten it!

```
LAB_WEEK12> db.posts.aggregate([ { $lookup: { from: "users", localField: "postedBy"
, foreignField: "_id", as: "author" } }, { $unwind: "$author" }] )
[
  {
    _id: 'P189',
    caption: 'admin admin admin',
    postedAt: ISODate('2025-09-18T00:00:00.000Z'),
    postedBy: ObjectId('690acaa698ed38792463b112'),
    author: {
      _id: ObjectId('690acaa698ed38792463b112'),
      fullName: 'Sandy Bonfilio Yuvens',
      email: 'sandy.bonfilio@student.umn.ac.id',
      age: 20,
      phoneNumber: '089637073749',
      registeredAt: ISODate('2025-11-05T03:55:18.842Z'),
      displayName: 'sandyb',
      isOverLegalAge: 'No',
      addresses: [
        {
```

Now, what's left to do is tidying up the displayed fields in each document. How about displaying only the post's **`caption`** and author's **`displayName`**?

```
LAB_WEEK12> db.posts.aggregate([ { $lookup: { from: "users", localField: "postedBy"
, foreignField: "_id", as: "author" } }, { $unwind: "$author" }, { $project: { _id:
 0, caption: 1, authorDisplayName: "$author.displayName" } }] )
[
  { caption: 'admin admin admin', authorDisplayName: 'sandyb' },
  {
    caption: 'Discover, Develop, Dominate!',
    authorDisplayName: 'ppif.umn'
  },
  {
    caption: 'Bringing Your Tech Experience 2025',
    authorDisplayName: 'byte_umn'
  },
  { caption: '#TGIF', authorDisplayName: 'sandal' },
  {
    caption: "Infinite's Registration Is Now Open!",
    authorDisplayName: 'byte_umn'
  }
]
```

TODO: create a new collection called **`comments`**, populate it however you'd like, with **at least 2 or more comments in 2 distinct posts**. Here's an example:

```
LAB_WEEK12> db.comments.find().pretty()
[
  {
    _id: ObjectId('690ca5bf98ed38792463b11a'),
    postId: 'P895',
    userId: ObjectId('690acaa698ed38792463b112'),
    comment: '🔥🔥🔥',
    commentedAt: ISODate('2025-11-06T20:35:02.000Z')
  },
  {
    _id: ObjectId('690ca5bf98ed38792463b11b'),
    postId: 'P895',
    userId: ObjectId('690a19509111cfc65f63b112'),
    comment: 'woooo!',
    commentedAt: ISODate('2025-11-06T13:42:23.616Z')
  },
  {
    _id: ObjectId('690ca5bf98ed38792463b11c'),
    postId: 'P895',
    userId: ObjectId('690a199b9111cfc65f63b113'),
    comment: 'ditunggu ya!',
    commentedAt: ISODate('2025-11-06T13:42:23.616Z')
  },
  {
    _id: ObjectId('690ca5bf98ed38792463b11d'),
    postId: 'P001',
    userId: ObjectId('690a1e179111cfc65f63b11b'),
    comment: 'menyalaaaa',
    commentedAt: ISODate('2025-08-10T00:00:00.000Z')
  },
  {
    _id: ObjectId('690ca5bf98ed38792463b11e'),
    postId: 'P001',
    userId: ObjectId('690a1e179111cfc65f63b11a'),
    comment: 'developer paling keren',
    commentedAt: ISODate('2025-08-11T00:00:00.000Z')
  },
  {
    _id: ObjectId('690ca5bf98ed38792463b11f'),
    postId: 'P189',
    userId: ObjectId('690a1e179111cfc65f63b117'),
    comment: 'kiw kiw',
    commentedAt: ISODate('2025-10-10T00:00:00.000Z')
  }
]
```